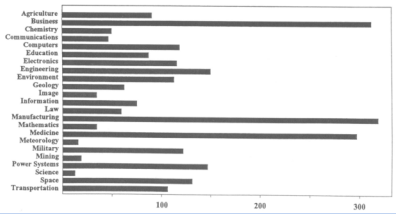
Les systèmes experts

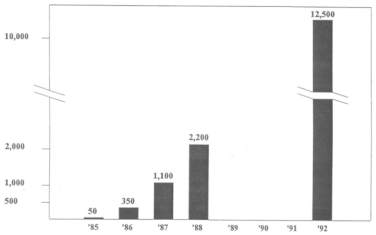
Ordre 0, Ordre 1.
Systèmes experts et Prolog.
La question en question.

Domaines d'application et leurs importances

Agriculture	Environment	Meteorology
Business	Geology	Military
Chemistry	Image Processing	Mining
Communications	Information Management	Power Systems
Computer Systems	Law	Science
Education	Manufacturing	Space Technology
Electronics	Mathematics	Transportation
Engineering	Medicine	



Nombre cumulés de SE développés



Types et répartition des problèmes traités

Problem Type	Description
Control	Governing system behavior to meet specification
Design	Configuring Objects under constraint
Diagnosis	Inferring System malfunctions from observables
Instruction	Diagnosing, debugging, and repairing student behavior
Interpretation	Inferring situation description from data
Monitoring	Comparing observations to expectations
Planning	Designing actions
Prediction	Inferring likely consequences of given situations
Prescription	Recommending solution to system malfunction
Selection	Identifying best choice from a list of possibilities
Simulation	Modelling the interaction between system components

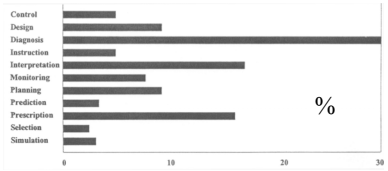
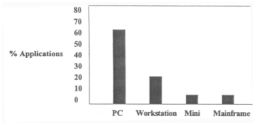
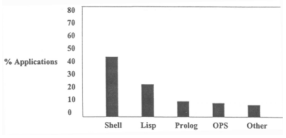


Plate-formes et langages de développement





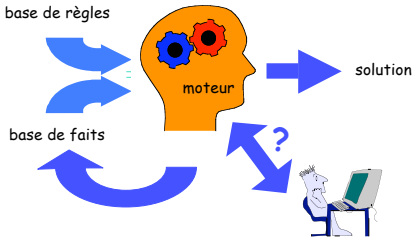
Introduction

Objectif affiché :
remplacer les experts humains d'un domaine ou les aider dans leurs expertises

Domaines d'application : nombreux et variés

Principe :
On dispose de connaissances sur un domaine et d'un moteur d'inférences. À partir d'informations contextuelles factuelles, on infère de nouvelles connaissances sur le problème examiné.

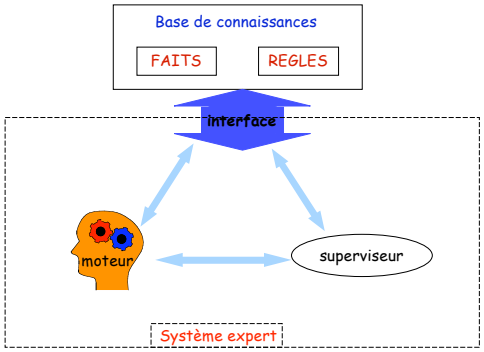
Principe



Constituants

Un système expert est composé de :

- **la base de connaissances** :
  - **base de faits** : code la connaissance sur l'étude en cours
    Son état évolue en cours d'expertise (*mémoire de travail*)
  - **base de règles** : code la connaissance sur le domaine.
    Fixe pour plusieurs expertises.
    Règle : **SI** *condition* **ALORS** *action*
- **le moteur d'inférences** :
  - composés des algorithmes utilisés pour la déduction :
    - chaînage avant
    - chaînage arrière
    - calcul des faits déductibles
    - calcul de question
    - chaînage mixte



Caractéristiques

- les connaissances sur le domaine peuvent être représentées de manière
  - finie (booléen, symbole, nombre)
  - incertaine ou floue
- la programmation de l'expertise est déclarative
- indépendance entre moteur d'inférences et base de connaissances
  systèmes experts essentiels
  ➔ séparation du procédural et du déclaratif
- justification du raisonnement
- recherche sur un but précis ou pas
- données monotones ou pas
- interrogation de l'utilisateur

Classification des systèmes experts

3 classes :

- **ordre 0**
  fondé sur le calcul propositionnel.
  Les faits sont à valeurs booléennes.
  SI *agé_de_plus_de_18_ans* **ALORS** *majeur*
- **ordre 0+ ou 0.5**
  les faits peuvent être à valeurs réelles ou symboliques
  SI *age* ≥ 18 **ALORS** *statut=majeur*
- **ordre 1**
  basés sur le calcul des prédicats
  utilisation des variables et de l'unification
  SI *age(X,Age)* **ET** *Age* ≥ 18 **ALORS** *majeur(X)*

Quelques systèmes experts

Spécifiques

- **Dendral** (1965) détermination de composés chimiques à partir de données spectrométriques
- **Mycin** (1970) expertise en bactériologie (diagnostic et traitement)

- **Prospector** (1978) évaluation géologique de sites (probabilité de présence de gisements)

Générateurs

- Guru
- SNARK
- EMycin
- CLIPS (NASA)
- Nexpert Object (Neuron Data)

Principe de réalisation

- trouver un expert ou plusieurs (c'est plus fiable) sur le domaine visé par l'expertise

- choisir un générateur de systèmes experts et déterminer les contraintes sur la représentation des connaissances (variables ? valeurs floues ? etc.) en fonction des possibilités

- interroger l'expert en lui demandant d'exprimer les valeurs clés de ses analyses et les règles d'inférences ingénieur cogniticien

- construire la base de règles

- tester

Représentation des connaissances

Base de Faits
et
Base de Règles

les faits

- 3 états possibles (**métavaleurs**) :
  - **CONNU** : une valeur a été attribuée (par déduction ou par l'utilisateur)
  - **INCONNU** : aucune information connue
  - **INDETERMINEE** : pas de valeurs connue et l'utilisateur a dit "*je ne sais pas*"
- 3 transitions possibles généralement (inférences monotones) :

INCONNU → CONNU INDETERMINE → CONNU
INCONNU → INDETERMINE

Si non monotonie, on peut avoir : CONNU → INCONNU

- des options : demandable et affichable, ouvert ou fermé
- des variantes :
  - coefficients de vraisemblance (*maladie* = *rougeole* (0.6))
  - introduction de variables (ordre 1)
  - valeurs floues (*taille* = *grande*)

les règles

SI conjonction de littéraux/prémisses **ALORS** conclusion/action

Exemple de base de règles (ordre 0)

SI fleur et graine ALORS phanérogame
SI phanerogame et graine nue ALORS sapin
SI phanerogame et 1-cotylédone ALORS monocotylédone
SI phanerogame et 2-cotylédone ALORS dicotylédone
SI monocotylédone et rhyzome ALORS muguet
SI dicotylédone ALORS anémone
SI monocotylédone et non rhyzome ALORS lilas
SI feuilles et fleur ALORS cryptogame
SI cryptogame et non racine ALORS mousse
SI cryptogame et racine ALORS fougère
SI non feuilles et plante ALORS thallophyte
SI thallophyte et chlorophylle ALORS algue
SI thallophyte et non chlorophylle ALORS champignon
SI non feuilles et non fleur et non plante ALORS colibacile

Utilisation de variables

permet une compacité des connaissances

Base composée de nombreux faits

père (?,??)

sans variable :

"autant" de faits

grand-père (?,???)

avec variable :

une régle

SI *père*(X,Y)
ET *père*(Y,Z)
ALORS *grand-père* (X,Z)

Problème : complexité pour le filtrage des règles par la base de faits

Conjonctions et disjonctions

- ET en conclusion :

SI *a* ALORS *b* ET *c* ≡ SI *a* ALORS *b*
SI *a* ALORS *c*

- OU en condition :

SI *a* OU *b* ALORS *c* ≡ SI *a* ALORS *c*
SI *b* ALORS *c*

Disjonctions en conclusion

- OU en conclusion : plus problématique

SI *a* ALORS *b* OU *c*

quelle sémantique ? dans le cas où *a* est *vrai*, que faire ?

– dédoubler la base de faits : l'une avec *b*, l'autre avec *c*
– ajouter *b* et *c* ? (alors il fallait mettre un ET)
– ajouter *b* puis backtrack avec *c* (si déduction impossible ou recherche d'une autre solution)

Généralement quand un expert exprime une telle régle : OU = XOR

et alors : SI *a* ALORS *b* OU *c* ≡ SI *a* ET ¬*b* ALORS *c*
SI *a* ET ¬*c* ALORS *b*

Analyse de la base de règles

Elle permet :

- classification des faits :
  – de base : n'apparaissent qu'en condition (généralement *demandable*)
  – intermédiaire : apparaissent en condition et conclusion de règles
    ils représentent les étapes intermédiaires du raisonnement et permettent de l'expliquer
  – conclusifs : n'apparaissent qu'en partie conclusion
    se sont les résultats possibles de l'expertise
- prévision d' "efficacité"
  – plus le nombre de prémisses dans une règle est important, plus la déduction de nouveaux faits risque d'être longue (base *large*)
  – plus le nombre de règles permettant de passer d'un fait de base à un fait conclusif est élevé, plus l'expertise risque d'être longue (base *haute*)

outils : graphes de dépendance arbres ET/OU

Choix des faits conclusifs

Cas d'un SE d'ordre 0 permettant de déterminer en fonction de critères le type de logement convenant le mieux à un client.
Les différents buts possibles sont à choisir parmi :
F2 *pavillon* *fermette* etc.
Il faut donc effectuer une expertise avec chacun des différents buts possibles, ce qui revient à poser les questions :
"est-ce que *F2* convient ?" "est-ce que *pavillon* convient ?" etc.

Autre solution : ajouter un fait conclusif, *trouve*
et les règles : SI *F2* ALORS *trouve* SI *pavillon* ALORS *trouve* etc.
et effectuer les expertises avec comme but *trouve*,
quand le but a été prouvé, il ne reste qu'à examiner la base de faits afin de connaître le type de logement

Le moteur d'inférences

Chaînages avant, arrière et mixte

Deux mécanismes de base

- chaînage avant

basé sur le modus ponens :

SI ($A \Rightarrow B$ ET A) ALORS B

- chaînage arrière

basé sur le modus tollens :

SI ($A \Rightarrow B$ ET $\neg B$) ALORS $\neg A$

Chaînage avant

Principe

à partir d'un état de la base de faits, effectuer toutes les déductions logiquement possibles

saturation de la base de faits

exemple :

BR= SI *a* ALORS *c*
SI *a* ET *d* ALORS *e*
SI *b* ET *c* ALORS *d*
SI *e* ALORS *f*

et $BF_0 = \{ a , b \}$

alors on peut déduire : *c* *d* *e* *f*

Mécanisme

on cherche les règles dont la condition est vérifiée et on applique leurs conclusions

Plusieurs étapes :

- sélection
  suppression de règles (ex : définitivement non déclenchables ou concluant sur un fait déjà établi)
- déterminaison des règles déclenchables (condition satisfaite)
- résolution de conflits : choisir la règle à déclencher selon *heuristiques*
- activation : appliquer la conclusion

Arrêt lorsque :

- il n'y a plus de règles déclenchables
- le but recherché a été établi (pas dans un "vrai" chaînage avant)

Un algorithme de chaînage avant (0+)

activer toutes les règles de la base
Tant qu'il existe des règles actives déclenchables
  choisir R une de ces règles ← résolution de conflits
  Si la conclusion de R ne contredit pas la base de faits
  alors
    appliquer la conclusion de R et désactiver R
  sinon
    base inconsistante STOP
  fin Si
fin Tant que

La désactivation des règles assure la terminaison de l'algorithme.

En ordre 1, on ne peut **pas** désactiver les règles.

SI $pair(X)$ ALORS $pair(s^2(X))$ ⟶ $BF_\infty = \{pair(s^{2n}(0)), \forall\ n \geq 0\}$

et $BF_0 = \{pair(0)\}$

Résolution de conflits

Problème du choix de la règle déclenchable à activer

Différentes heuristiques possibles :

- dans l'ordre de l'écriture de la base
- au hasard
- toutes les règles déclenchables simultanément
- à partir de coefficients de priorité sur les règles
- règle dont la condition utilise les faits les plus récemment déduits
- à partir de règles de contrôle (méta-règles)

Exemple

R1 SI fleur et graine ALORS phanérogame
R2 SI phanerogame et graine nue ALORS sapin
R3 SI phanerogame et 1-cotylédone ALORS monocotylédone
R4 SI phanerogame et 2-cotylédone ALORS dicotylédone
R5 SI monocotylédone et rhyzome ALORS muguet
R6 SI dicotylédone ALORS anémone
R7 SI monocotylédone et non rhyzome ALORS lilas
R8 SI feuilles et fleur ALORS cryptogame
R9 SI cryptogame et non racine ALORS mousse
R10 SI cryptogame et racine ALORS fougère
R11 SI non feuilles et plante ALORS thallophyte
R12 SI thallophyte et chlorophylle ALORS algue
R13 SI thallophyte et non chlorophylle ALORS champignon
R14 SI non feuilles et non fleur et non plante ALORS colibacile

BF_0={rhyzome, fleurn graine, 1-cotylédone, feuilles, non racine}

Résolution des conflits : ordre d'écriture des règles **BF_∞ ?**

Choix de la résolution de conflits ?

Théorème :

En fonctionnement monotone, soient BR une base de règles d'ordre 0+, sans métavaleur et BF_0 une base de faits, alors :
- quel que soit l'ordre des prémisses dans les règles
- quelle que soit la méthode de résolution des conflits
  ≡ quel que soit l'ordre des règles

BR sature BF_0 en la même base BF_∞.

- Si non monotone : remise en question possible des déductions

R_1 : SI a ALORS retirer b $R2$: SI b ALORS retirer a

$BF_0 = \{a,b\} \xrightarrow{R_1} BF_1 = \{a\} = BF_\infty$

et

$BF_0 = \{a,b\} \xrightarrow{R_2} BF_1 = \{b\} = BF_\infty$

- si métavaleurs

R_1 : SI a INCONNU ALORS b $R2$: SI c ALORS a

$BF_0 = \{c\} \xrightarrow{R_1} BF_1 = \{c,b\} \xrightarrow{R_2} BF_2 = \{a,b,c\} = BF_\infty$

et

$BF_0 = \{c\} \xrightarrow{R_2} BF_1 = \{c,a\} = BF_\infty$

- si ordre 1

R_1 : SI $pair(X)$ ALORS $pair(s^2(X))$ $R2$: SI $pair(X)$ ALORS $impair(s(X))$

- on déclenche dans l'ordre d'écriture des règles :

$BF_0 = \{c\} \xrightarrow{R_1^*} BF_\infty = \{pair(s^{2n}(0)), \forall\ n \geq 0\}$

- on déclenche simultanément toutes les règles déclenchables :

$BF_0 = \{c\} \xrightarrow{\{R_1, R_2\}^*} BF_\infty = \{pair(s^{2n}(0)), impair(s^{2n+1}(0)), \forall\ n \geq 0\}$

Remarques

Avec chaînage avant :

- pas besoin de but
- pénalisé par beaucoup de règles avec les mêmes prémisses
- déduit tout ce qui peut l'être
- nécessite de dispose dès le début de toutes les données (pas d'interaction avec l'utilisateur)
- à l'issue d'une saturation, il est possible que certains ne soient plus déductibles :

R_1 : SI a ALORS $\neg\, b$ $\qquad$ $R2$: SI b ALORS c

$$BF_0 = \{a\} \xrightarrow{R_1} BF_1 = \{a, \neg\, b\} = BF_\infty$$

c n'est plus déductible

Algorithme de calcul des faits déductibles

déclarer déductibles tous les faits connus et inconnus demandables
activer toutes les règles
Tant qu'il existe des règles actives telles que :
leur condition n'a aucune prémisses fausses et
a chacune de ses prémisses déductibles ou à métavaleur
déclarer déductibles les faits de la conclusion
désactiver la règle
fin Tant que

Cet algorithme calcule en fait un sur-ensemble des faits déductibles.

SI a ALORS b
SI $\neg a$ ALORS c
SI b et c ALORS d

l'algorithme déclare d déductible

Mais il est suffisant pour des raisons d'efficacité

Chaînage arrière

Principe

On recherche les connaissances nécessaires à la preuve d'un but donné. On considère les règles qui ont le but pour conclusion, si la condition de l'une de ces règles est vérifiée alors succès, sinon les prémisses inconnues deviennent les nouveaux (sous-)buts.

exemple :

$BR=$ SI a ALORS c
SI a ET d ALORS e
SI b ET c ALORS d
SI e ALORS f

et $\quad BF = \{ a, b \}$

$but = f$

Deux méthodes

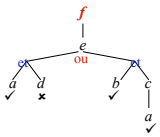
arbres ET/OU

$BR=$ SI a ALORS c
SI a ET d ALORS e
SI b ET c ALORS e
SI e ALORS f

et $\quad BF = \{ a, b \}$

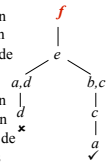
arbre de listes de buts
(gestion par pile des buts)

$but = f$



Si une branche d'un nœud ET donne un échec, il est inutile de vérifier les autres

Si une branche d'un nœud OU donne un succès, il est inutile de calculer les autres



Résolution de conflits

- choix de la règle concluant sur le but
  - dans l'ordre d'écriture dans la base
  - au hasard
  - celle avec le moins de prémisses inconnues
  - coefficient de priorité
  - méta-règles
- choix parmi les prémisses du fait devenant le nouveau but
  - ordre d'écriture dans la règle
  - demandables en priorité
  - non demandables en priorité

Exemple

SI a ET b ET c ALORS d
SI i ET h ALORS b
SI a ET f ALORS b
SI a ALORS i
SI e ET f ALORS d
SI a ALORS f
SI k ET l ALORS e
SI a ALORS l

$BF = \{a,k\}$

$but = d$

Remarques

Avec chaînage arrière :

- guidé par un but
- pénalisé par beaucoup de règles avec la même conclusion
- n'utilise que les règles nécessaires
- nécessite de disposer dès le début de toutes les données

Chaînage mixte

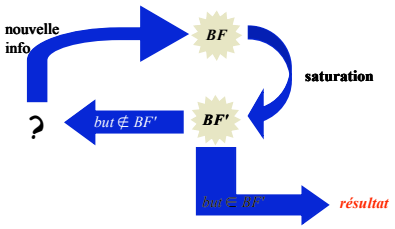
Constat :

Les chaînages avant et arrière nécessitent une connaissance suffisante dès le début. Si les connaissances initiales sont insuffisantes, la conclusion recherchée peut ne pas pouvoir être établie.

Solution :

Le système doit pouvoir poser des questions à l'utilisateur en cours de raisonnement afin de compléter ses connaissances, notamment pour les informations non calculables par déduction.

Principe



Poser une question

Le système ne peut poser des questions que sur les faits demandables.

Le choix de la question posée est un critère pour mesurer la qualité (l'intelligence) d'un SE.

Principe

effectuer un chaînage arrière jusqu'à atteindre un but demandable

Les réponses possibles sont:

VRAI , FAUX , valeur,

"je ne sais pas" (INCONNU)

Un algorithme

poser_question(*but*) : (*but* supposé déductible)

fait courant = *but*
Tant que *fait courant* non demandable
  déterminer une règle concluant sur *fait courant* dont chaque prémisse est : satisfait, déductible ou méta-valeur
  déterminer dans cette règle un fait inconnu
  fait courant = ce fait
fin Tant que
poser une question sur *fait courant*

résolutions de conflits

choix "classiques" pour les résolutions des conflits :

- **règle** : celle ayant le moins de prémisses inconnus
- **fait** : on préfère les non demandables

SI *a* ET *b* ALORS *d*
SI *c* ALORS *ï*

but : d

Algorithme de chaînage mixte

poser_question(*but*) :

effectuer une saturation
calculer les faits déductibles
Tant que *but* est INCONNU et déductible
  poser_question(*but*)
  effectuer une saturation
  calculer les faits déductibles
fin Tant que
Si *but* CONNU
  alors donner la valeur de *but*
  sinon *but* non déductible
fin Si

Exemple

SI *g* ET *h* ALORS *k*
SI *j* ALORS *k*
SI *c* ET *b* ALORS *g*
SI *a* ET *c* ET *d* ALORS *h*
SI *e* ET *a* ALORS *j*
SI *a* ALORS *b*

Faits demandables = faits de base

Résolution de conflits pour question :
choix de règle et fait dans l'ordre d'écriture

$BF_0=\{d\}$ *But* : *k*

- que se passe-t-il si *a* devient *Faux* ?
- que donne *chaînage_mixte*(*k*) ?
  (réponses : *NON* puis toujours *OUI*)

Exemple

R1. SI *a* ET *b* ET *c* ET *d* ALORS *Conc*
R2. SI *u* ET *v* ALORS *Conc*
R3. SI *w* ET *z* ET *y* ALORS *Conc*
R4. SI *j* ALORS *z*
R5. SI *j* ALORS *a*
R6. SI *j* ALORS *b*
R7. SI *j* ALORS *d*
R8. SI *f* ET *g* ALORS *c*
R9. SI *a* ET *k* ALORS *c*
R10. SI *w* ALORS *k*
R11. SI *j* ET *e* ALORS *g*

$BF_0=\{\neg u\}$

But : *Conc*

Questions :
Faux pour *w*,
Vrai pour les autres.

- chaînage avant : on parcourt les règles dans leur ordre d'écriture dans la base
- chaînage arrière, on préfère :
  - sur les règles : les règles avec de moins de prémisses inconnues
  - sur les faits : les faits non demandables.

Dans les deux cas, en situation d'égalité on prend l'ordre d'écriture.